

L'écosystème des DataFrames en Python

Les librairies, le fichier Parquet, les formats de tables — les bases avant l'atelier.

Au programme

01	Fondations	DataFrame, colonnes, Arrow
02	Les librairies	Pandas, Polars, DuckDB, Dask, PySpark
03	Le fichier Parquet	colonnes, pages, footer, pushdown
04	Delta Lake et Iceberg	du dossier de fichiers à la table

01

Fondations

Ce que toutes ces librairies ont en commun avant de diverger.

Qu'est-ce qu'un DataFrame

date date32	ville utf8	temp float64	capteur int32
2026-01-04	Namur	3.4	18
2026-01-04	Liège	1.9	42
2026-01-05	Namur	4.1	18

Une table nommée, en mémoire, dont chaque colonne porte un type unique.

Colonne = tableau typé

Chaque colonne est un bloc contigu d'un seul type, pas une liste d'objets Python.

Opérations vectorisées

On décrit l'opération sur la colonne entière ; la boucle vit dans du code compilé.

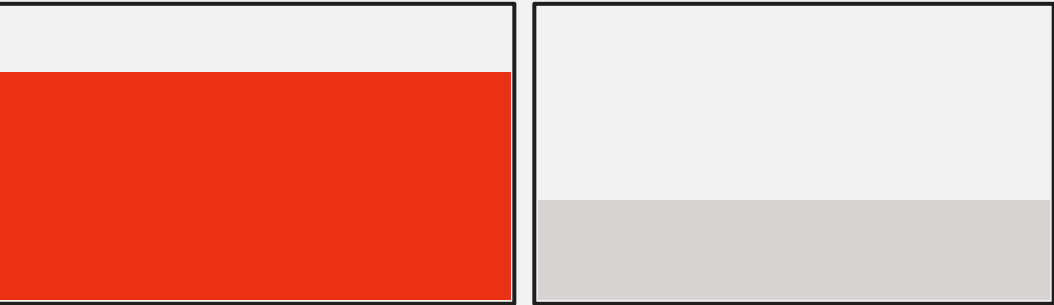
Schéma explicite

Noms et types sont connus avant le calcul : c'est ce qui rend l'optimisation possible.

Les trois murs d'un DataFrame en mémoire

MUR 01

La mémoire



Un jeu de 5 Go sur disque peut demander 3 à 10 fois plus en RAM une fois décompressé et converti.

MUR 02

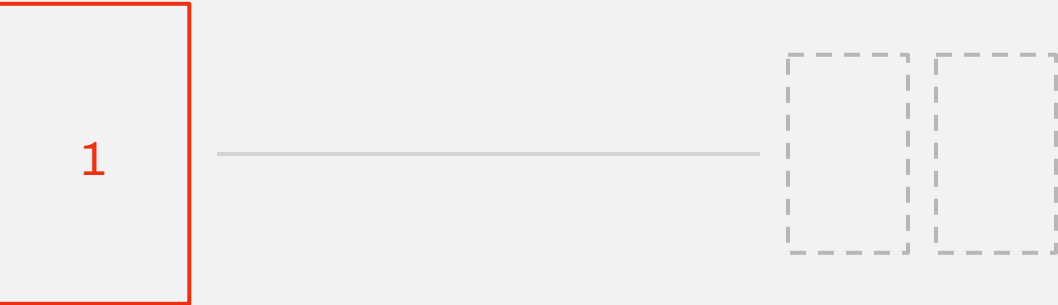
Le cœur unique



Le modèle historique exécute sur un seul thread : sept cœurs sur huit restent inutilisés.

MUR 03

La machine unique



Au-delà du plus gros serveur disponible, il faut découper le travail sur plusieurs machines.

Ligne par ligne ou colonne par colonne

Orienté ligne — CSV, JSON, base transactionnelle

3 accès dispersés pour une colonne

date	ville	temp	capt.	date	ville	temp	capt.	date	ville	temp	capt.
------	-------	------	-------	------	-------	------	-------	------	-------	------	-------

Une ligne complète est contiguë. Idéal pour lire ou écrire un enregistrement entier.

Orienté colonne — Parquet, Arrow, entrepôts analytiques

1 accès contigu

date	date	date	ville	ville	ville	temp	temp	temp	capt.	capt.	capt.
------	------	------	-------	-------	-------	------	------	------	-------	-------	-------

Une colonne complète est contiguë, homogène, donc compressible et vectorisable.

Colonne ou ligne : les compromis

Ligne pour écrire des événements, colonne pour les analyser : les deux cohabitent dans presque toutes les architectures.

COLONNE · AVANTAGES

- On ne lit que les colonnes utilisées, quelle que soit la largeur de la table.
- Valeurs homogènes : compression et encodages très efficaces.
- Calcul vectorisé : une boucle serrée sur un bloc contigu du même type.

COLONNE · INCONVÉNIENTS

- Reconstituer une ligne entière demande d'assembler N colonnes.
- Modifier une valeur suppose de réécrire un bloc entier, sinon le fichier.

LIGNE · AVANTAGES

- Lire ou écrire un enregistrement complet coûte un seul accès.
- Insertion et mise à jour ligne par ligne, sans réécrire de bloc.
- Écriture en flux continu, adaptée à l'ingestion et aux journaux.

LIGNE · INCONVÉNIENTS

- Une moyenne sur une colonne traverse tout le fichier.
- Types mélangés dans un bloc : compression médiocre, aucun pruning.

Ajouter une ligne : l'avantage de l'orienté ligne

Orienté ligne

1 écriture

ligne 1	04-01 · Namur · 3.4 · 18
ligne 2	04-01 · Liège · 1.9 · 42
ligne 3	04-02 · Namur · 4.1 · 18
ligne 4	04-02 · Gand · 2.7 · 91

La ligne est déjà contiguë : on l'écrit à la suite, sans toucher au reste du fichier. C'est le mode d'écriture d'un journal, d'un CSV, d'une base transactionnelle.

Orienté colonne

4 blocs à rouvrir

date	ville	temp	capt.
04-01	Namur	3.4	18
04-01	Liège	1.9	42
04-02	Namur	4.1	18
04-02	Gand	2.7	91

Les quatre valeurs de la même ligne vivent dans quatre blocs éloignés, chacun compressé et encodé à part : il faut les rouvrir, les décompresser et les réécrire tous les quatre.

D'où l'écriture par paquets

Parquet n'ajoute jamais une ligne : il écrit un row group entier, et le fichier est scellé par son footer.

D'où les petits fichiers

Un flux de mille insertions par heure produit mille fichiers minuscules si personne ne compacte.

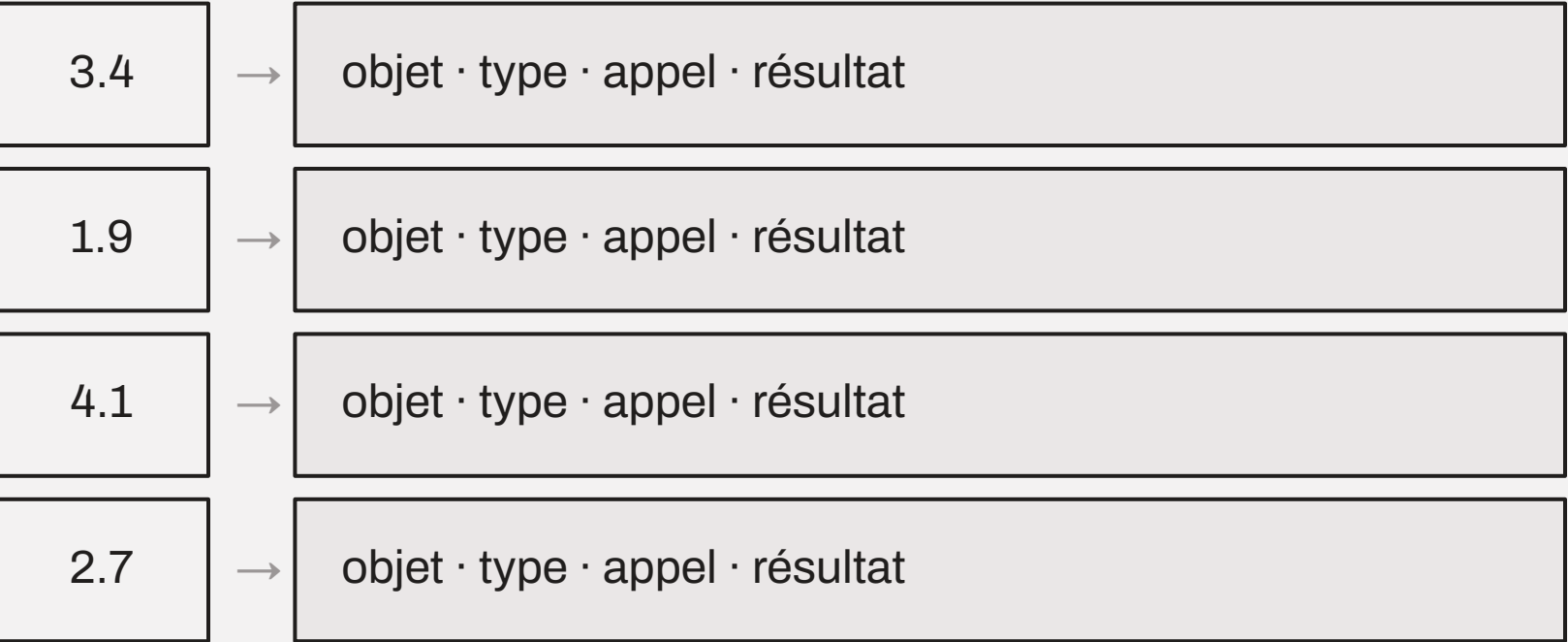
D'où Delta et Iceberg

Ajouter des fichiers plutôt que des lignes, puis compacter — c'est exactement ce que ces formats organisent.

Ce que veut dire « vectorisé »

Boucle en Python

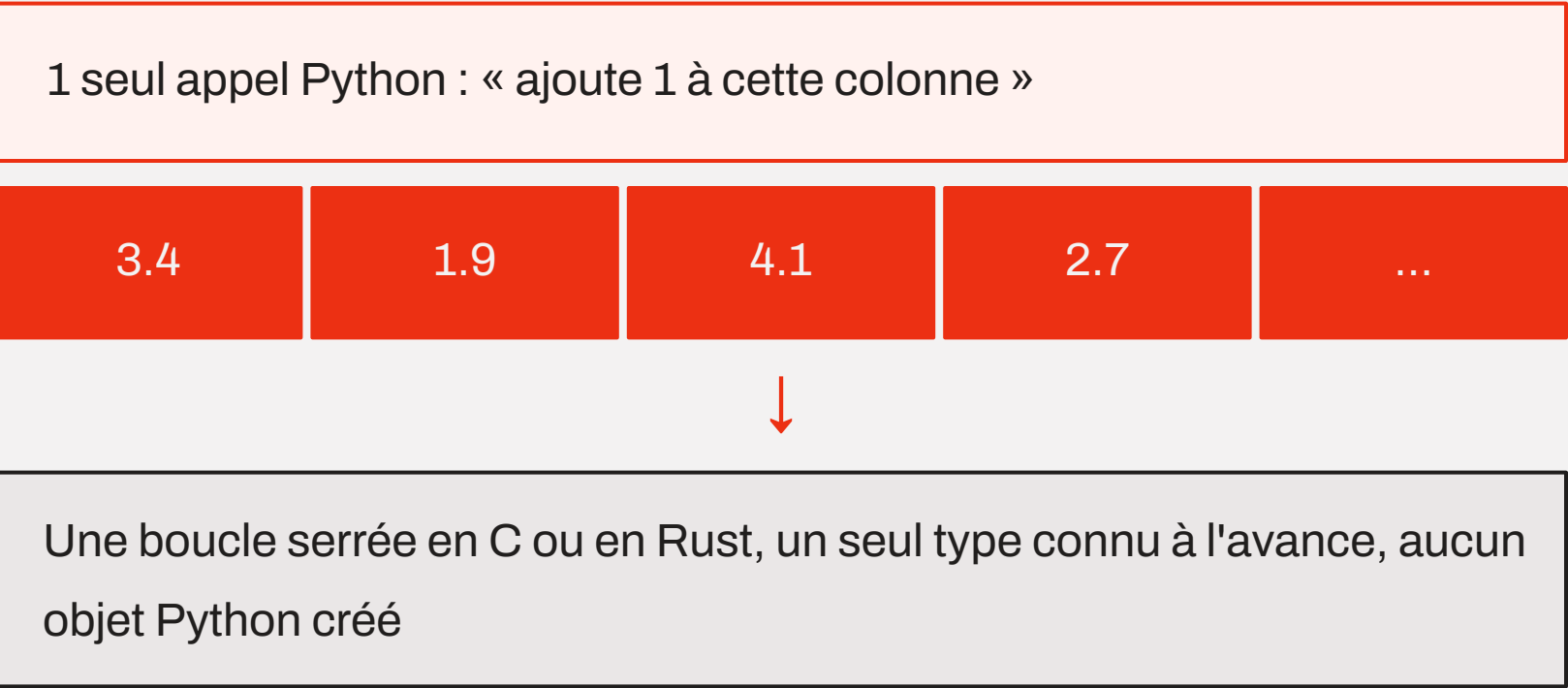
1 valeur à la fois



L'interpréteur refait tout le protocole à chaque valeur : déballer l'objet, vérifier le type, appeler l'opération, allouer le résultat. Le calcul utile est une fraction infime du travail.

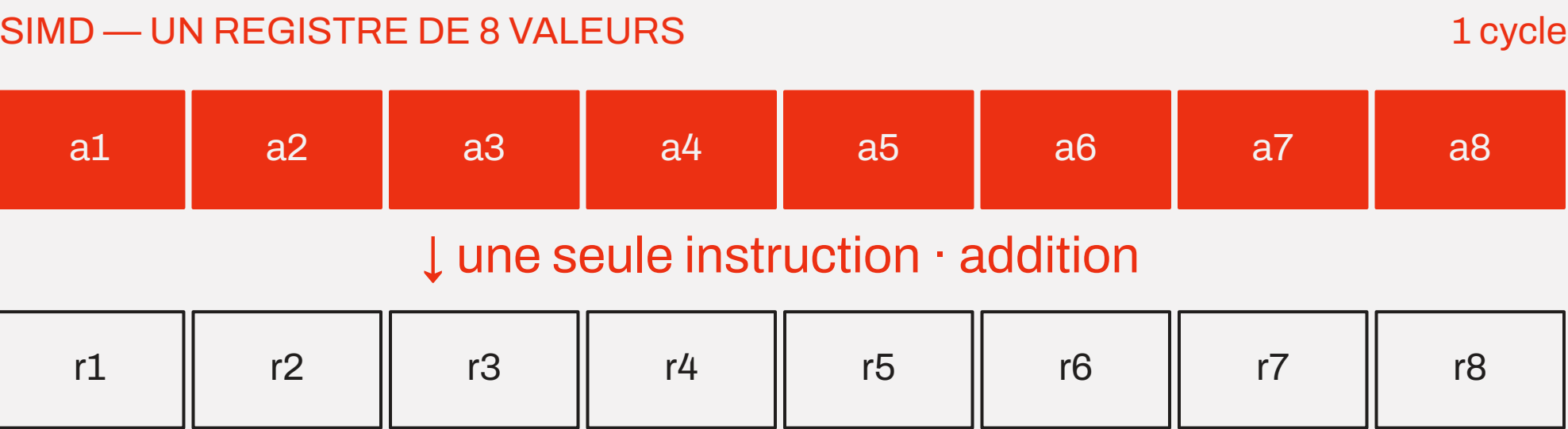
Opération vectorisée

1 appel, N valeurs



Le type est vérifié une fois pour toute la colonne. C'est cette absence de protocole par valeur qui donne les rapports de 10 à 100.

SIMD — Single Instruction Multiple Data



AVX2 traite 256 bits d'un coup, soit 8 entiers de 32 bits ou 4 flottants de 64 bits ;
AVX-512 et NEON suivent le même principe à d'autres largeurs.

Pourquoi le colonne est la condition

Remplir un registre suppose des valeurs du même type, côte à côte en mémoire. C'est exactement la définition d'une colonne Arrow — et l'inverse d'une ligne, où quatre types se succèdent.

Ce qui empêche le SIMD

Des objets Python au lieu de valeurs brutes, des types mélangés, une donnée dispersée en mémoire, ou une opération dépendant du résultat de la précédente.

Les trois étages se cumulent

SIMD dans le cœur, plusieurs cœurs dans la machine, plusieurs machines dans le cluster — dans cet ordre, du gain le moins coûteux au plus coûteux.

Apache Arrow, le format mémoire commun

DEPUIS 2016

Arrow

Une spécification de représentation colonne en RAM : buffers de valeurs, buffer de validité, métadonnées de schéma.

Un langage commun.

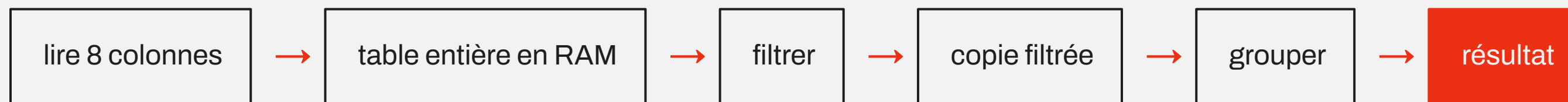
Polars natif Arrow	DuckDB échange zero-copy
pandas 2.x backend Arrow optionnel	PySpark transferts JVM ↔ Python
cuDF même modèle sur GPU	Parquet lecture via pyarrow

Deux moteurs qui parlent Arrow s'échangent une table en **zero-copy** : on passe une adresse mémoire, pas une copie.

Eager et lazy : deux moments d'exécution

Eager — chaque appel calcule tout de suite

pandas



Chaque étape matérialise un résultat intermédiaire. Facile à déboguer, coûteux en mémoire.

Lazy — les appels décrivent, l'exécution vient à la fin

Polars, DuckDB, Dask, PySpark



L'optimiseur réordonne, fusionne, pousse les filtres vers le fichier et ne lit que les colonnes utiles.

Le même calcul, deux plans

LA QUESTION **Température moyenne par ville en mars 2026**

Source : mesures.parquet — 12 Go, 40 colonnes, 12 mois, un row group par mois.

Eager

exécuté dans l'ordre écrit

LIRE le fichier — 40 colonnes, 12 mois	12 Go lus
MATÉRIALISER la table en mémoire	≈ 20 Go RAM
FILTREER mars 2026	copie n° 1
SÉLECTIONNER ville, temp	copie n° 2
GROUPER par ville, moyenne(temp)	résultat

Le filtre arrive après la lecture : le moteur ne pouvait pas savoir qu'il ne servirait qu'un mois et deux colonnes.

Lazy — plan optimisé

réordonné avant lecture

GROUPER par ville, moyenne(temp)	résultat
SCAN mesures.parquet	
projection : ville, temp	2 / 40 colonnes
predicate : date en mars 2026	1 / 12 row groups
Octets réellement lus	90 Mo

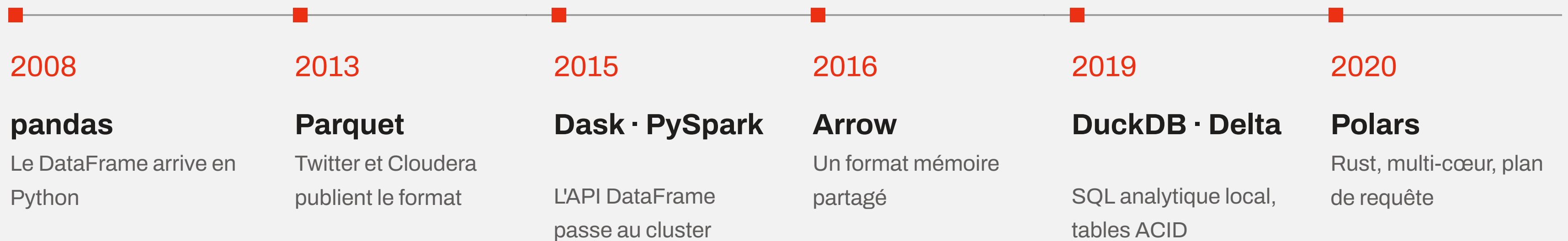
Le filtre et la sélection remontent dans le scan, l'agrégat consomme le flux : aucune table intermédiaire n'existe.

02

Les librairies

Cinq réponses aux mêmes trois murs, à quinze ans d'écart.

Quinze ans d'écosystème



Aucune de ces librairies n'a remplacé la précédente : elles cohabitent parce qu'elles répondent à des contraintes différentes.

pandas — la référence historique

ORIGINE	2008, Wes McKinney, chez AQR Capital
ÉCRIT EN	Python, Cython et C, au-dessus de NumPy
EXÉCUTION	Eager, mono-thread, tout en mémoire
SIGNATURE	L'index : chaque ligne porte une étiquette, les opérations s'alignent dessus
DEPUIS 2.0	Backend Arrow optionnel, types nullable, copy-on-write

Le vocabulaire commun de la donnée en Python.

Quinze ans de tutoriels, de recettes et d'intégrations : c'est encore le point d'entrée de tout le reste de l'écosystème, et la cible de conversion de toutes les autres librairies.

pandas — avantages et limites

Avantages

- Écosystème sans équivalent : scikit-learn, matplotlib, statsmodels, Jupyter.
- API très riche pour le nettoyage, les séries temporelles, le pivot, le reshape.
- Retour immédiat : idéal pour explorer et apprendre.
- Lit à peu près tout : CSV, Excel, SQL, JSON, Parquet, HDF5.

Limites

- Mono-thread et pas d'optimiseur de requête.
- Empreinte mémoire élevée, résultats intermédiaires copiés.
- API à plusieurs chemins pour un même besoin ; l'index surprend souvent.
- Le mur arrive vite : quelques Go et le poste sature.

QUAND L'UTILISER

Jeux de données de quelques centaines de Mo, exploration en notebook, dernière étape avant un modèle ou un graphique.

Polars — le moteur écrit en Rust

ORIGINE	2020, Ritchie Vink, projet indépendant
ÉCRIT EN	Rust
EXÉCUTION	Deux API : eager et lazy, avec optimiseur et moteur streaming
SIGNATURE	Les expressions : <code>col()</code> composable, parallélisée sur tous les cœurs
PAS D'INDEX	Un DataFrame est une liste ordonnée de colonnes nommées, rien d'autre

Le confort du DataFrame, la discipline d'un moteur de requêtes.

On écrit du Python, mais tout le calcul se passe dans un moteur compilé qui voit la requête entière avant de commencer à lire.

Polars — avantages et limites

Avantages

- Multi-thread par défaut, sans configuration.
- Le mode lazy applique projection et predicate pushdown jusqu'au fichier.
- API cohérente et typée : une seule façon d'exprimer une opération.
- Le moteur streaming traite des volumes plus grands que la RAM (out-of-core).

Limites

- Écosystème plus jeune : moins de bibliothèques l'acceptent en entrée directe.
- Réapprentissage réel pour qui a des réflexes pandas.
- Une seule machine : pas de cluster. En bêta on prem/AWS
- API encore mouvante d'une version mineure à l'autre.

QUAND L'UTILISER

Transformations de 1 à 100 Go sur une seule machine, pipelines de production, remplacement direct d'un traitement pandas trop lent.

DuckDB — le SQL analytique en processus

ORIGINE	2019, CWI Amsterdam — laboratoire d'où vient aussi MonetDB
ÉCRIT EN	C++
EXÉCUTION	Vectorisée
INTERFACE	SQL — et une API relationnelle Python
SIGNATURE	Interroge directement des fichiers Parquet, CSV, JSON, locaux ou sur S3

Un entrepôt analytique de la taille d'une librairie.

Pas de serveur à installer, pas de données à charger : la requête va chercher les fichiers là où ils sont, et lit un DataFrame pandas ou Polars comme une table.

DuckDB — avantages et limites

Avantages

- SQL complet : jointures, window functions, CTE récursives, agrégats.
- Excellent lecteur Parquet : pushdown, partitions, globs, S3.
- Échange zero-copy avec Arrow, pandas et Polars dans le même processus.
- Extensions pour Delta, Iceberg, PostgreSQL, géospatial.

Limites

- Un seul processus, une seule machine : pas de calcul distribué.
- Écrire du SQL n'est pas toujours pratique pour un enchaînement de transformations.
- Conçu pour l'analytique, pas pour des écritures transactionnelles concurrentes.
- Le va-et-vient SQL / DataFrame casse un peu le fil du code.

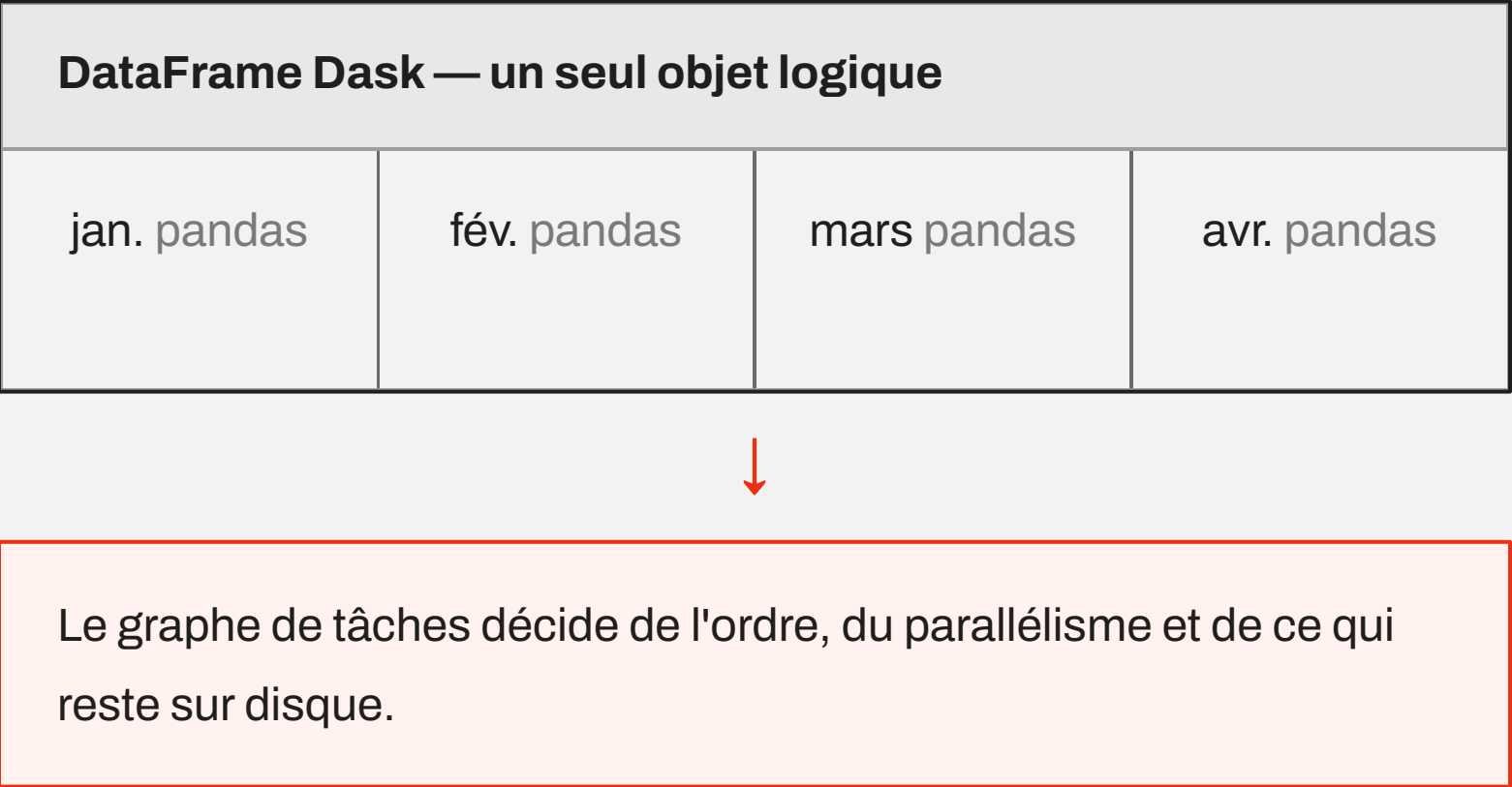
QUAND L'UTILISER

Agrégations et jointures sur un data lake en Parquet, exploration ad hoc, remplacement d'un entrepôt pour des volumes moyens.

Dask — pandas partitionné

ORIGINE	2015, Matthew Rocklin, Continuum Analytics
ÉCRIT EN	Python pur, orchestrant pandas et NumPy
EXÉCUTION	Graphe de tâches paresseux, threads, processus ou cluster
SIGNATURE	L'API pandas, presque à l'identique, à l'échelle

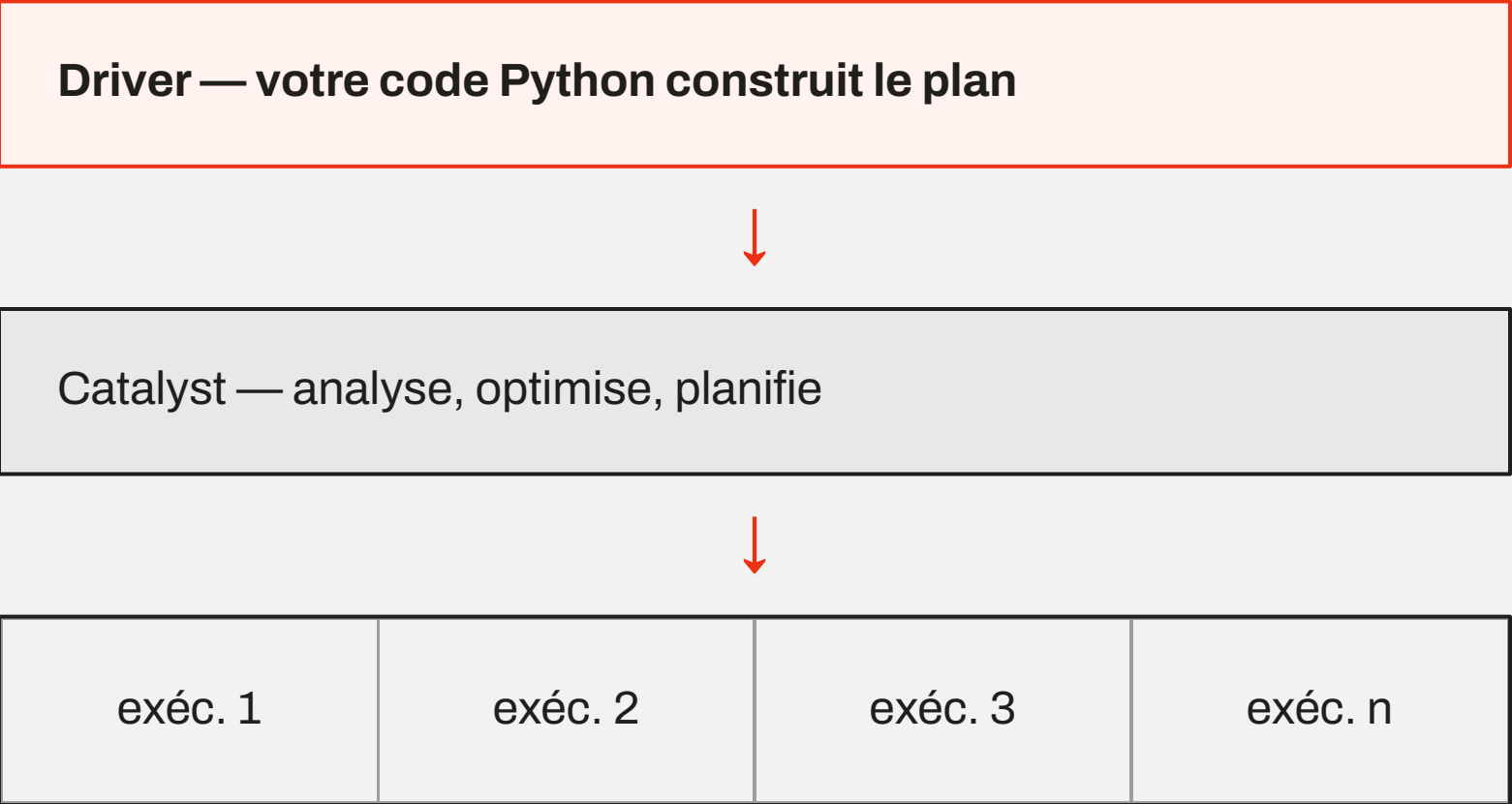
Chaque partition est un DataFrame pandas complet. Une opération est appliquée partition par partition, puis les résultats sont recombines.



PySpark — le calcul distribué

ORIGINE	Spark, 2009 à Berkeley ; API DataFrame en 2015
ÉCRIT EN	Scala
EXÉCUTION	Lazy, plan optimisé par Catalyst, exécuté sur N exécuteurs
SIGNATURE	Le même code du PC portable au cluster

Tant qu'on reste dans l'API DataFrame ou SQL, le code Python ne fait que construire un plan : rien ne traverse la frontière Python / JVM.



Les données ne remontent au driver que sur un collect() ou un toPandas().

Dask et PySpark — avantages et limites

Dask

- Tout en Python : même langage, mêmes outils de débogage.
- Migration douce depuis un code pandas existant.
- Sert aussi à paralléliser du code qui n'est pas un DataFrame.
- Hérite des faiblesses de Pandas.
- Le choix des partitions conditionne tout ; les shuffles coûtent cher.

PySpark

- Passe réellement au To et au Po, avec tolérance aux pannes.
- Écosystème mature : SQL, streaming, ML, Delta, catalogues.
- Standard de fait des plateformes managées.
- JVM à installer et à régler ; démarrage lent, messages d'erreur opaques.
- Surdimensionné en dessous de quelques dizaines de Go.

À RETENIR

Le distribué ne se choisit pas pour la vitesse mais parce que le volume ne tient plus sur une machine.

Le reste de l'écosystème

cuDF · RAPIDS

Le même modèle colonne, exécuté sur GPU NVIDIA.

Modin

Remplacement direct de l'import pandas, exécuté sur Ray ou Dask. Zéro réécriture, gains variables.

Vaex

Fichiers mappés en mémoire et colonnes calculées à la volée, pour explorer de très grandes tables localement.

Ibis

Une API DataFrame qui se compile en SQL pour vingt moteurs : on écrit une fois, on exécute où l'on veut.

Narwhals

Couche de compatibilité pour écrire une librairie qui accepte indifféremment pandas, Polars ou PyArrow.

Ray Data · Daft

Traitement distribué orienté charges d'apprentissage et données multimodales, en Python et Rust.

Le point commun de presque tous : Arrow en mémoire, Parquet sur disque.
C'est là que l'interopérabilité se joue.

Tableau comparatif

Librairie	Origine	Écrit en	Exécution	Échelle confortable
Pandas	2008	C / Cython, NumPy	eager, 1 cœur	< 1 Go
Polars	2020	Rust	eager + lazy, N cœurs	1 Go à 1 To
DuckDB	2019	C++	SQL vectorisé, N cœurs	1 Go à 1 To
Dask	2015	Python	graphe de tâches	10 Go à quelques To
PySpark	2015	Scala	lazy, cluster	100 Go et au-delà

Les échelles sont indicatives : la forme des données et la mémoire disponible déplacent les frontières.

Ordres de grandeur sur un group-by

Agrégation sur 10 Go de Parquet, poste à 8 cœurs et 32 Go de RAM. Temps relatif, le plus rapide valant 1.



L'écart ne vient pas du langage seul : il vient du multi-cœur, du pushdown et de l'absence de copies intermédiaires.

Quelle librairie pour quel cas

Exploration, petit volume, sortie vers un graphique ou un modèle	→	Pandas
Pipeline de transformations, une machine, code Python maintenable	→	Polars lazy
Requêtes, jointures et agrégats sur un dossier de fichiers Parquet	→	DuckDB
Base de code Pandas existante à passer à l'échelle sans réécrire	→	Dask
Volumes qui ne tiennent sur aucune machine, cluster et catalogue en place	→	PySpark

Dans un même projet, deux d'entre elles cohabitent très bien : Arrow rend le passage de l'une à l'autre presque gratuit.

03

Le fichier Parquet

Le format pour l'analyse

Les limites du CSV

CSV	Parquet
Aucun type : tout est du texte, à inférer à chaque lecture	Schéma typé écrit dans le fichier
Orienté ligne : lire une colonne = lire tout	Orienté colonne : on lit ce qu'on demande
Compression du fichier entier, indécoupable	Compression par page, encodage adapté au type
Aucune métadonnée : il faut tout lire pour savoir	Statistiques min / max / nulls par bloc
Lisible par un humain, écrit par tout outil	Binaire : il faut une librairie pour l'ouvrir

Un jeu de 10 Go de CSV pèse couramment 1 à 2 Go en Parquet — et la requête n'en lit qu'une fraction.

Les quatre idées de Parquet

Publié en 2013 par Twitter et Cloudera, d'après le modèle de stockage décrit par Google dans Dremel.

01

Découper en colonnes

Les valeurs d'une même colonne sont écrites ensemble, donc lisibles seules.

02

Découper en blocs

Les lignes sont regroupées par paquets, ce qui rend le fichier parallélisable.

03

Décrire le contenu

Schéma, positions et statistiques sont écrits à la fin, dans le footer.

04

Encoder finement

Chaque page choisit l'encodage et la compression adaptés à ses valeurs.

CONSÉQUENCE

Un moteur peut décider quoi lire — et surtout quoi ne pas lire — avant d'ouvrir la moindre donnée.

Anatomie d'un fichier Parquet

PAR1	Row group 1				Row group 2				Row group n				Footer schéma · index · stats	4 o longueur	PAR1
	date	ville	temp	capteur	date	ville	temp	capteur	date	ville	temp	capteur			

en-tête

Les données, groupées par blocs de lignes puis par colonne

Les métadonnées, écrites en dernier

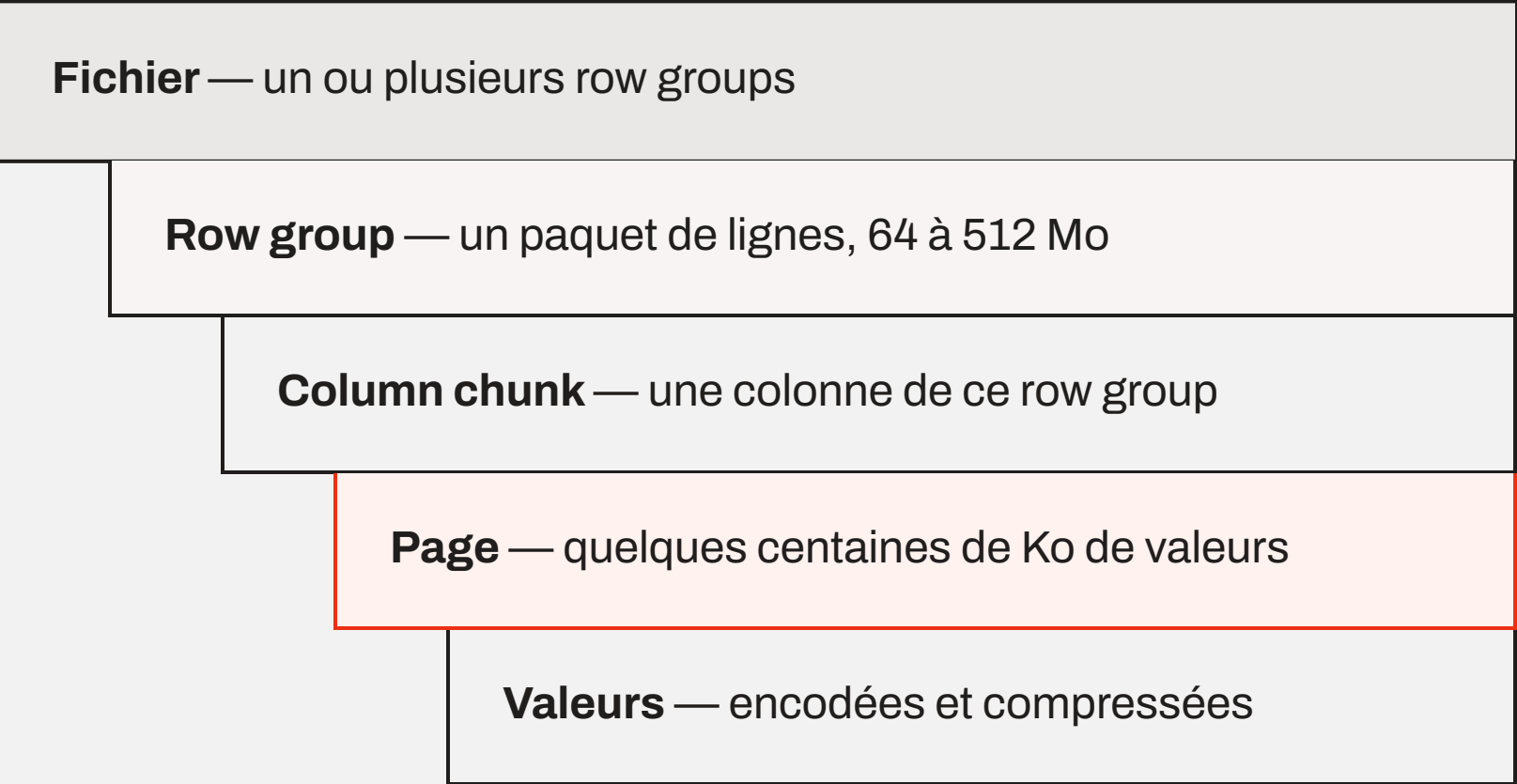
Pourquoi les métadonnées à la fin

Les statistiques et les positions ne sont connues qu'une fois les données écrites. Les mettre à la fin permet d'écrire le fichier en une seule passe, sans revenir en arrière.

Le lecteur commence par la fin

Il lit les 8 derniers octets, y trouve la taille du footer, le charge, puis saute directement aux octets utiles.

Du row group à la page



Le row group est l'unité de parallélisme

Un moteur donne un row group par cœur ou par tâche. Trop petits, les fichiers deviennent bavards ; trop gros, le filtrage devient grossier.

La page est l'unité de décodage

On ne décompresse jamais moins qu'une page. Chaque page porte son propre en-tête, son encodage et, en option, ses statistiques.

Trois sortes de pages

La page de dictionnaire ouvre le chunk, les pages de données suivent ; les définition et répétition levels encodent les valeurs nulles et les structures imbriquées.

Le footer et ses métadonnées

Footer — métadonnées du fichier
Version du format et schéma complet, colonne par colonne
Nombre total de lignes
Pour chaque row group : nombre de lignes, taille
Pour chaque colonne d'un row group : position dans le fichier, taille compressée, encodages, codec
Statistiques : min, max, nombre de nulls, valeurs distinctes
En option : page index, Bloom filters
Métadonnées libres écrites par le producteur

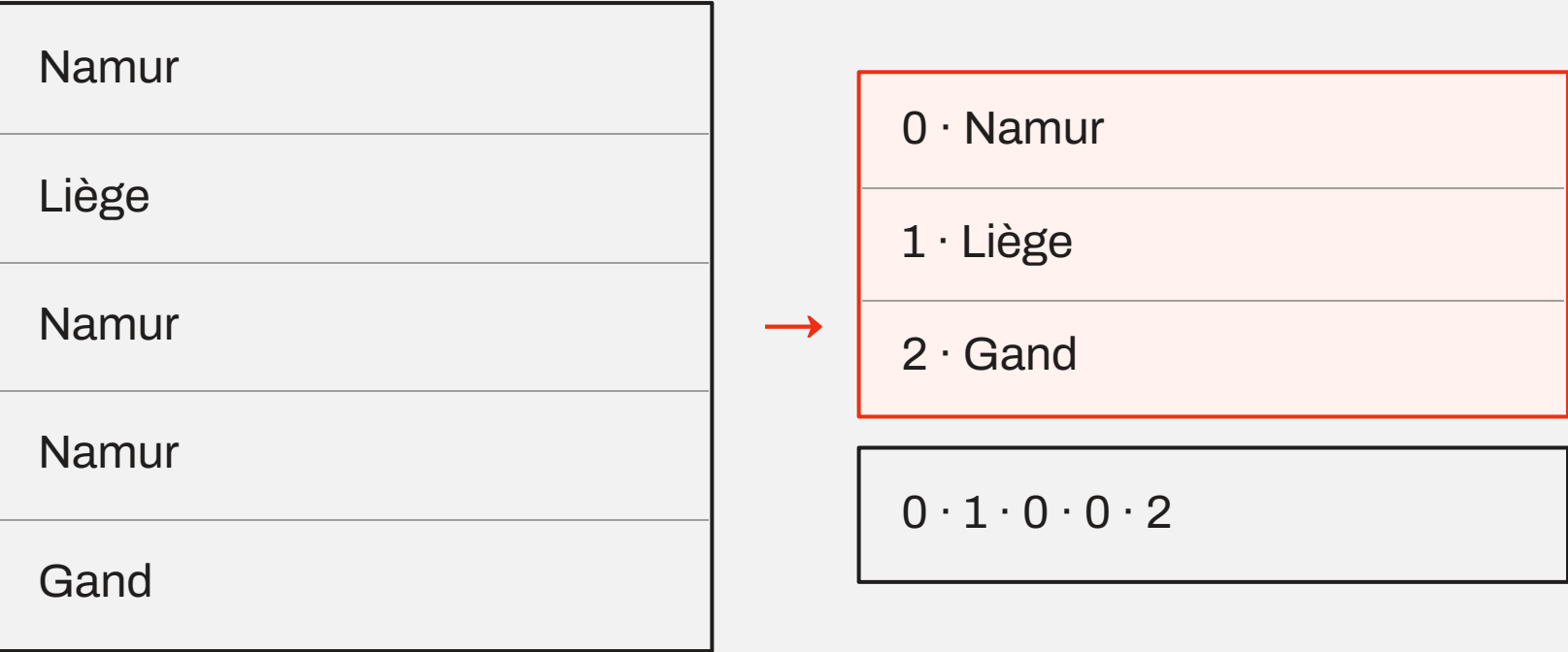
L'ordre de lecture

- 01Lire les 4 derniers octets : PAR1, le fichier est valide
- 02Lire les 4 octets précédents : la longueur du footer
- 03Charger le footer et y lire schéma, positions et statistiques
- 04Décider quels row groups et quelles colonnes méritent d'être lus
- 05Sauter directement aux pages retenues et les décoder

Sur un stockage objet, ces cinq étapes ne coûtent que deux ou trois requêtes réseau au lieu du fichier entier.

Encodages et compression

Encodage par dictionnaire



Le texte répété devient une table de valeurs plus une suite de petits entiers. C'est l'encodage par défaut des colonnes à faible cardinalité.

Run-length et bit-packing

Vingt fois la même valeur s'écrit « 20 × 0 », et les petits entiers n'occupent que les bits nécessaires.

Delta et byte stream split

Les horodatages et compteurs croissants ne stockent que l'écart ; les flottants se compressent mieux octet par octet.

Puis la compression du bloc

Snappy pour la vitesse, zstd pour le meilleur compromis, gzip par héritage — page par page, jamais le fichier entier.

Une colonne homogène se compresse bien mieux qu'une ligne hétérogène.

Le predicate pushdown

FILTRE DEMANDÉ **date ≥ 2026-03-01**

Row group 1	Row group 2	Row group 3	Row group 4	Row group 5
min 01-01 max 01-31	min 02-01 max 02-28	min 03-01 max 03-31	min 04-01 max 04-30	min 01-15 max 02-10
ignoré	ignoré	lu	lu	ignoré

Comment le moteur décide

Si le max d'un bloc est inférieur au seuil, aucune de ses lignes ne peut satisfaire le filtre : le bloc est écarté sans être lu. Trois blocs sur cinq disparaissent ici avant tout accès aux données.

Ce qui le rend efficace

Des données triées sur la colonne filtrée. Sur une colonne dispersée au hasard, tous les blocs contiennent tout : les statistiques ne servent plus à rien.

Quatre niveaux de pruning

Le même filtre est appliqué quatre fois, de plus en plus finement, avant qu'une seule valeur ne soit comparée.

01 · Partition	Le nom du dossier porte la valeur : des dossiers entiers sont écartés sans ouvrir un fichier	chemins
02 · Row group	Le min / max du bloc est incompatible avec le filtre : le bloc est ignoré	stats du footer
03 · Page	À l'intérieur d'un bloc retenu, seules les pages compatibles sont décompressées	page index
04 · Valeur	Ce qui reste du filtre s'applique aux lignes décodées, en mémoire	le moteur

Le Bloom filter, en renfort

Sur une égalité, un Bloom filter écrit dans le fichier répond « certainement absent » sans lire la colonne — utile là où min / max ne discrimine rien.

Chaque niveau divise le suivant

Les gains se multiplient : un dossier sur douze, un bloc sur dix, une page sur cinq — et il ne reste presque rien à décoder.

Le projection pushdown

COLONNES DEMANDÉES **ville, temp**

id	date	ville	temp	capteur	humidité	vent	commentaire
non lu	non lu	lu	lu	non lu	non lu	non lu	non lu

Le footer donne les positions

Chaque column chunk a un offset et une taille : le lecteur saute au bon endroit du fichier, sans traverser les autres colonnes.

Le piège du select *

Une seule colonne texte volumineuse peut représenter la moitié du fichier. La demander sans en avoir besoin annule tout le bénéfice du format.

Ce qui empêche le pushdown

Une fonction appliquée à la colonne

Filtrer sur `année(date) = 2026` empêche la comparaison aux statistiques ; filtrer sur un intervalle de dates la permet.

Des données non triées

Si les valeurs sont réparties au hasard, chaque bloc va du minimum au maximum global : aucun bloc n'est éliminable.

Le mode eager

Charger puis filtrer en Python ne pousse rien : le fichier a déjà été lu en entier. C'est tout l'intérêt de scan puis filtre.

Le select * et le collect trop tôt

Demander toutes les colonnes, ou matérialiser avant d'avoir filtré, annule la projection comme le predicate.

CONSÉQUENCE
PRATIQUE

Trier les données à l'écriture sur la colonne que l'on filtrera est souvent le gain de performance le moins coûteux.

Les deux pushdowns se multiplient

Une table de 40 colonnes, 12 mois de données, 2 colonnes demandées sur 1 mois.

12 Go Le fichier complet sur disque	1 Go Après projection : 2 colonnes sur 40	90 Mo Après predicate : 1 mois sur 12	0,7 % Des octets du fichier réellement lus
---	---	---	--

C'est aussi pour cela que la lecture directe sur stockage objet est devenue viable : on ne télécharge presque rien.

Partitionnement et pièges classiques

Le partitionnement par dossiers

mesures/
annee=2025/mois=11/ — part-0.parquet
annee=2025/mois=12/ — part-0.parquet
annee=2026/mois=01/ — part-0.parquet
annee=2026/mois=02/ — part-0.parquet

Le chemin porte la valeur : un filtre sur l'année ou le mois élimine des dossiers entiers avant même d'ouvrir un fichier. Partitionner sur une colonne à forte cardinalité produit en revanche des milliers de dossiers minuscules.

Quatre pièges fréquents

- Des milliers de fichiers de 2 Mo : le coût est dans les ouvertures, pas dans les octets. Viser 128 Mo à 1 Go par fichier.
- Row groups trop gros : le filtrage devient grossier et la mémoire de lecture explose.
- Données non triées sur la colonne filtrée : les statistiques min / max ne discriminent plus rien.
- Schémas divergents d'un fichier à l'autre : la lecture du dossier échoue ou perd des colonnes.

Ces quatre points sont exactement ceux que Delta Lake et Iceberg prennent en charge.

04

Delta Lake et Iceberg

Passer d'un dossier de fichiers à une véritable table.

Un dossier de Parquet n'est pas une table

Pas d'atomicité

Un travail d'écriture interrompu laisse des fichiers à moitié écrits, que les lecteurs voient déjà.

Pas de mise à jour

Corriger une ligne suppose de réécrire les fichiers concernés, sans garantie pour qui lit au même moment.

Pas d'historique

Impossible de relire la table telle qu'elle était hier, ni de savoir ce qui a changé depuis.

Pas de liste fiable

Savoir ce que contient la table demande de parcourir tous les dossiers du stockage : lent, coûteux, et jamais cohérent.

LE MANQUE

Ce qui manque n'est pas un moteur de plus, mais une définition de ce qui appartient à la table à un instant donné.

Une couche de métadonnées transactionnelle

pandas	Polars	DuckDB	PySpark	Trino, Flink...
--------	--------	--------	---------	-----------------



Format de table — Delta Lake ou Iceberg

Quels fichiers composent la table maintenant, avec quel schéma, dans quelle version



part-0.parquet	part-1.parquet	part-2.parquet	part-3.parquet	...
----------------	----------------	----------------	----------------	-----

Les données restent des fichiers Parquet ordinaires, sur un stockage objet ordinaire. Ce qui change, c'est qu'un fichier ne compte que s'il est référencé par la version courante.

Delta Lake — le journal de transactions

ma_table/_delta_log/	
...0000.json	ajoute part-0, part-1
...0001.json	ajoute part-2
...0002.json	retire part-0, ajoute part-3
...0010.checkpoint.parquet	état complet condensé
_last_checkpoint	où reprendre la lecture
ma_table/*.parquet	les données, inchangées

Lire la table = rejouer le journal depuis le dernier checkpoint pour obtenir la liste exacte des fichiers valides.

Origine

Créé par Databricks pour Spark, ouvert en 2019. Journal en JSON, checkpoints en Parquet.

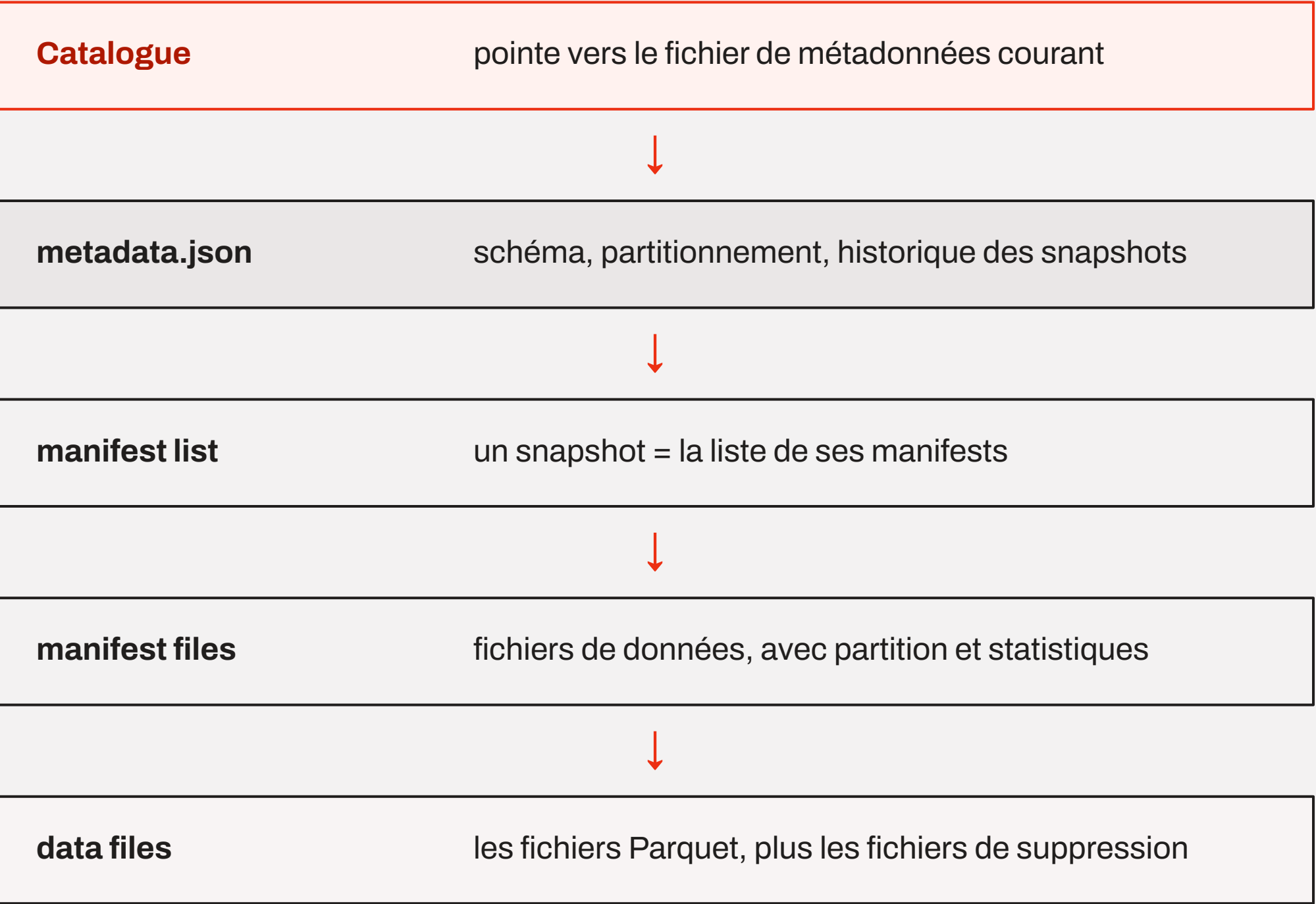
Écrire

Optimistic concurrency : deux écritures visent le même numéro de commit, une seule gagne, l'autre rejoue son travail.

En Python

Le paquet deltalake, écrit en Rust, lit et écrit des tables Delta sans Spark ; Polars et DuckDB s'y branchent.

Apache Iceberg — l'arbre de métadonnées



Origine

Créé chez Netflix en 2017 pour des tables de plusieurs pétaoctets, confié à la fondation Apache l'année suivante.

Hidden partitioning

La table connaît sa règle de partitionnement : filtrer sur une date suffit. La règle peut changer sans réécrire l'historique.

Le catalogue est central

C'est lui qui rend le changement de version atomique : REST, Glue, Nessie, Hive.

Ce que les deux formats apportent

Transactions ACID

Une écriture est visible entièrement ou pas du tout, même interrompue, même à plusieurs.

Time travel

Relire la table à une version ou une date, comparer, revenir en arrière après une erreur.

Mises à jour et suppressions

Update, delete et merge directement sur les fichiers, sans tout réécrire à la main.

Évolution du schéma

Ajouter, renommer ou réordonner une colonne sans casser les lecteurs ni réécrire les données.

Pruning à grande échelle

Les statistiques sont centralisées dans les métadonnées : plus besoin de parcourir le stockage pour planifier.

Maintenance outillée

Compaction des petits fichiers, tri, expiration des anciennes versions : des opérations prévues par le format.

Delta et Iceberg comparés

	Delta Lake	Iceberg
ORIGINE	Databricks, 2019	Netflix, 2017, puis Apache
MÉTADONNÉES	Journal de commits JSON, checkpoints Parquet	Arbre de fichiers Avro immuables par snapshot
VERSION COURANTE	Le dernier fichier du journal	Le pointeur du catalogue
PARTITIONNEMENT	Dossiers, ou clustering déclaré sur la table	Caché et modifiable sans réécriture
TERRAIN DE JEU	Écosystème Spark et Databricks	Moteurs multiples : Trino, Flink, Snowflake, BigQuery
DEPUIS PYTHON	deltalake, Polars, DuckDB, PySpark	Pylceberg, DuckDB, Polars, PySpark

Le choix se fait sur la plateforme et le catalogue, rarement sur la fonctionnalité.

À retenir

- 01 Le choix d'une librairie est un choix de contrainte : volume, nombre de cœurs, nombre de machines.
 - 02 Le mode lazy n'est pas une optimisation cosmétique : c'est ce qui permet de ne pas lire.
 - 03 Parquet est colonne, découpé en blocs et décrit par son footer : d'où la projection et le predicate pushdown.
 - 04 Delta et Iceberg ajoutent au-dessus des fichiers ce qui manquait : transactions, historique, schéma vivant.
 - 05 Arrow et Parquet sont le terrain commun : c'est ce qui rend les combinaisons possibles.
-

Place à l'atelier

01

Lire le même Parquet avec pandas, Polars lazy et DuckDB, et comparer.

02

Ouvrir le footer d'un fichier et y lire schéma, row groups et statistiques.

03

Écrire une table Delta, la modifier, puis relire sa version précédente.